

skilltrend: Characterizing an Agentic LLM Pipeline for Job-Market Skill-Demand Analysis

Aaron Chen (aaronyc), UW CSE Systems for ML

Spring 2026

Abstract

We build and characterize **skilltrend**, an agentic application that collects job postings, extracts structured skills through concurrent LLM workers, applies deterministic validation, and computes time-windowed skill trends. On a fixed 100-posting benchmark using hosted **gemini-2.5-flash-lite**, throughput scales from 0.54 postings/s sequentially to 7.92 postings/s with 16 workers, a **14.7x speedup**, while per-call p50/p95 latency remains nearly flat. The hosted backend therefore absorbs our fan-out without observable queueing in the tested range. We also find that prompt-only negative constraints are insufficient: replaying the final blocklist over stored outputs removes 877 generic or non-actionable mentions across 452 of 2,446 extraction rows. The main practical optimization is consequently architectural: treat the LLM as a candidate generator and enforce output policy with cheap deterministic tools.

1. Introduction

Structured extraction agents differ from chat workloads in ways that directly affect system design. Job descriptions are long, outputs are compact JSON, and each posting can be processed independently. The workload is therefore input-heavy and highly parallelizable. Its usefulness also depends on negative constraints: broad terms such as **AI**, **SQL**, and **Cloud** should not outrank specific skills such as **PyTorch** or **BigQuery**, yet smaller models do not reliably obey such instructions.

This project asks: **how does client-side worker concurrency affect application throughput and per-call latency on a fixed extraction workload, and which optimizations most improve the resulting application?**

The application intentionally analyzes the market rather than matching candidates to jobs. It extracts each posting once, persists a normalized skill record, and answers many later trend queries without another model call. This separates the expensive ingestion path from the interactive analysis path and makes systems experiments repeatable.

We contribute:

1. An end-to-end agentic pipeline with 10 job-source providers, concurrent structured extraction, deter-

ministic validation, and reusable trend computation.

2. A controlled concurrency sweep over the same 100 postings at 1, 2, 4, 8, and 16 workers.
3. An analysis showing that hosted-backend concurrency scales well in our tested range, while deterministic post-processing is essential for actionable output.

2. System Design

skilltrend answers: *Which technical skills are gaining or losing share in recent job postings?* The current artifact contains **2,246 active postings across 24 companies**, collected from standard ATS APIs, Workday, Amazon Jobs, company-specific endpoints, and a JobSpy/LinkedIn fallback.

The application is an orchestrator-worker pipeline:

1. The scanner concurrently invokes source-specific provider tools and normalizes their outputs into one **Posting** schema.
2. The extraction orchestrator decomposes selected postings into independent LLM calls. An **asyncio.Semaphore(N)** bounds in-flight requests.
3. Each worker requests structured JSON containing role family, seniority, domain tags, and required/preferred skills with evidence.
4. A deterministic blocklist removes generic terms, soft skills, and work activities; a taxonomy normalizer canonicalizes aliases.
5. Trend computation joins persisted postings and extractions, then ranks skills by change in posting share between time windows.

The primary agentic pattern is **planning and decomposition with tool orchestration**. Providers are heterogeneous tools behind a common interface, and extraction workers operate concurrently over decomposed tasks. Deterministic validation is a refinement stage rather than LLM self-reflection: the current production path does not re-prompt the model.

The LLM client uses the OpenAI Chat Completions interface, allowing the same pipeline to target hosted APIs, vLLM, Ollama, or a deterministic offline test model. Experiments in this paper use Google’s OpenAI-compatible Gemini endpoint.

Source behavior is intentionally visible to the rest of the system only through normalized records. Green-

house and Ashby return descriptions inline; Workday and several company-specific providers require list/detail fan-out; and JobSpy wraps a synchronous crawler behind an asynchronous adapter. The largest current sources are Greenhouse (900 postings), Ashby (500), and Workday (282). Of the 2,246 postings, 114 lack provider-reported `posted_at`; trend computation uses `first_seen` for those records. This fallback enables immediate analysis but biases the first collection window, as discussed in Section 5.

Postings and extractions are persisted as CSV, while per-call metrics are append-only JSONL. This simple design is sufficient because each scan/extraction run writes in bulk, and it makes every paper result directly inspectable. The storage boundary also means that provider changes do not affect extraction logic and trend queries do not invoke the LLM. Figure 5 shows the resulting web dashboard.

3. Experimental Methodology

3.1 Hypotheses

We expected: **H1**, throughput would scale near-linearly until backend saturation; **H2**, saturation would appear as growing p95 latency; and **H3**, batching and repeated-prefix reuse would be important on a self-hosted backend.

3.2 Controlled Workload

Every configuration re-extracts the deterministic `head(100)` of `postings.csv`. Identical totals of 201,294 prompt tokens and 48,045 completion tokens across runs confirm that execution strategy, rather than workload drift, explains the differences.

We evaluate sequential execution (`workers=1`) and concurrent execution (`workers` in $\{2, 4, 8, 16\}$). The client uses `openai.AsyncOpenAI` with JSON response mode and disables its local rate limiter for the controlled paid-tier runs. Each call records latency, token counts, model, worker count, and success status; each run records wall-clock time, throughput, and p50/p95 latency.

The first `workers=8` run was a transient outlier, so we ran that configuration three times and report its median. Other configurations are single runs, a limitation discussed in Section 5.

Per-call latency measures the extraction worker from request start through parsing and deterministic normalization. Wall-clock time additionally captures scheduling and orchestration overhead across the entire run. Throughput is total selected postings divided by wall-clock time. The benchmark is intended to characterize the application-visible behavior of the hosted service; it cannot expose server-side queueing time, accelerator utilization, or batch composition.

Table 1: Controlled 100-posting concurrency sweep. The 8-worker row is the median of three runs.

Workers	Wall (s)	Posts/s	Speedup	p50 (ms)	p95 (ms)
1	184.65	0.54	1.0x	1641	3252
2	89.18	1.12	2.1x	1567	2975
4	45.62	2.19	4.1x	1613	3058
8	21.77	4.59	8.5x	1449	2932
16	12.63	7.92	14.7x	1557	3125

4. Results

4.1 Concurrency Scaling

Table 1 shows that throughput improves from 0.54 postings/s sequentially to 7.92 postings/s at 16 workers. All displayed configurations complete 100/100 calls successfully.

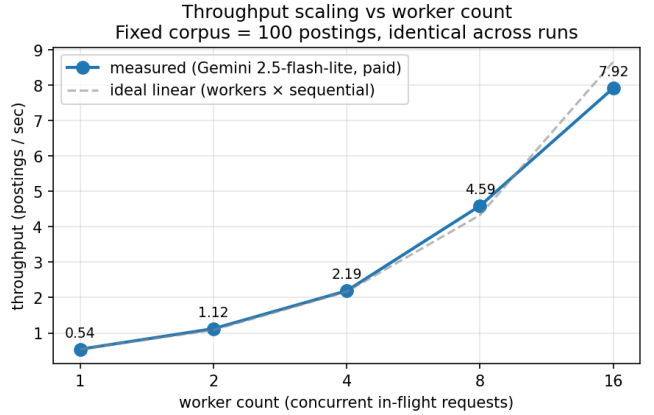


Figure 1: Throughput remains close to ideal linear scaling through 16 workers.

The throughput plot supports **H1**: scaling is near-linear, and the tested range does not reach saturation. At 16 workers, throughput is about 92% of ideal linear scaling relative to the sequential baseline.

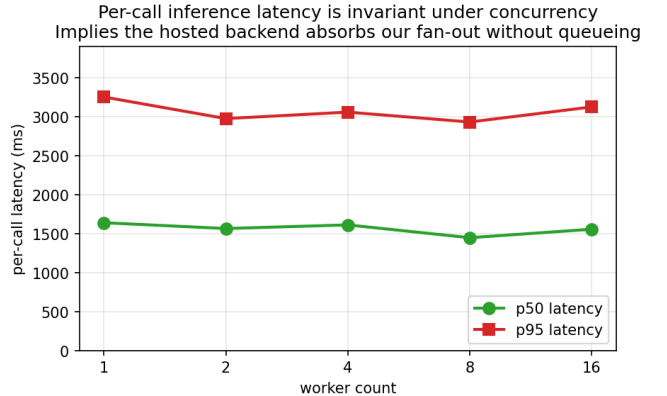


Figure 2: Per-call p50 and p95 latency remain nearly invariant as concurrency rises.

The latency plot rejects **H2** in the observed regime. p50 remains around 1.4-1.6 s and p95 around 2.9-3.3

s despite a 16x concurrency increase. Google’s hosted service either has enough internal parallelism for this workload or distributes requests across enough replicas that our client does not create observable queueing. This negative result matters: optimization opportunities on a hosted service differ from those of a self-hosted single replica.

Including the two extra 8-worker repetitions, all 700 calls in the controlled benchmark logs succeeded. The first 8-worker run took 88.75 s, compared with 21.77 s and 20.61 s on reruns, while its per-call p50/p95 remained close to the other runs. Per-call inference latency therefore does not explain the wall-clock outlier. The result demonstrates why hosted-API studies require repetition even when requests and token totals are fixed.

4.2 Workload Character

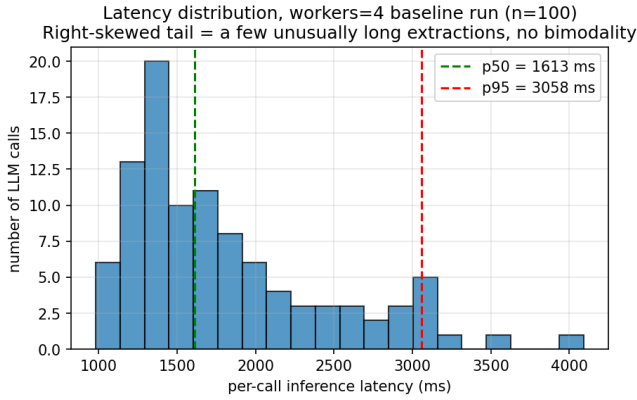


Figure 3: The 4-worker latency distribution is right-skewed without clear bimodality.

The 4-worker latency distribution is right-skewed but not clearly bimodal. Together with the flat percentiles across worker counts, this supports the interpretation that the observed tail comes from variable inference time on heterogeneous descriptions rather than increasing backend queueing.

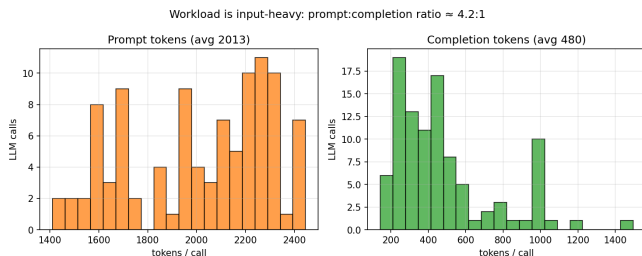


Figure 4: Prompt tokens dominate the controlled extraction workload.

Prompt tokens account for 80.7% of total tokens, with a prompt:completion ratio of approximately **4.2:1**. This confirms an input-heavy workload where repeated-prefix caching and efficient prefill could matter. However, the

hosted-client path does not expose server batching controls or cache-hit metrics, so **H3 remains a future vLLM experiment** rather than a measured optimization.

4.3 Application-Level Output Quality

The extraction prompt explicitly forbids umbrella terms and work activities. Nevertheless, early trend reports ranked items such as **AI**, **SQL**, **AI tools**, **negotiation**, and **French** as rising skills. Replaying the final blocklist over the current extraction table finds **877 blocked mentions across 452 of 2,446 rows (18.5%)**. Some rows predate later blocklist expansion, so this is not a clean final-model error rate, but it quantifies the amount of noise admitted by prompt-only extraction during development.

The production pipeline therefore applies the blocklist immediately after extraction and again during trend computation. Persisted normalized outputs also let users query new windows, companies, or categories without additional LLM calls.

The blocklist is deliberately narrow and auditable. It removes exact matches for generic technology categories, soft skills, human-language fluency, business activities, and work activities, then canonicalizes surviving aliases. This is more reliable and cheaper than asking the LLM to retry every questionable output. It also makes the final trend computation deterministic for a fixed posting/extraction corpus.

5. Discussion

5.1 Interpretation and Hosted-Backend Variance

H1 holds through the tested concurrency range, while **H2 does not**: the hosted backend does not visibly saturate at 16 in-flight requests. The result should not be generalized to a single self-hosted replica. A hosted service can route calls across an unknown number of machines, whereas vLLM-on-TPU would expose a controlled batching and queueing regime.

The anomalous first 8-worker run also limits the strength of the reported speedups. Reporting the median is preferable to selecting the fastest rerun, but repeating only the anomalous configuration creates asymmetric uncertainty. A final experiment should run every setting at least three times, randomize their order, and report error bars.

5.2 Optimization Opportunities

The largest demonstrated optimization is **deterministic post-processing**. It is negligible relative to an LLM call, removes non-actionable output, canonicalizes aliases, and turns extraction into a reusable ingestion-time cost.

The next systems experiment should deploy the same OpenAI-compatible client against vLLM on TPU. Unlike

Skill trends — all 24 companies · all categories

30d / vs prior 60d 90d / vs prior 90d 180d / vs prior 180d 365d / vs prior 365d

Current window — last 30d ⓘ

2813 postings

Baseline window — 30-90d ago (60d span) ⓘ

907 postings

Δ share (pp) = (skill_mentions_current / 2813) - (skill_mentions_baseline / 907), in percentage points. A skill can have *higher* count and *negative* Δ if the total posting mix changed.

Rising

Skill	Category	Mentions (of 2813)	Mentions (of 907)	Δ share (pp)
Java	Languages	406	31	11.02
C++	Languages	507	76	9.64
C#	Languages	247	5	8.23
JavaScript	Languages	218	26	4.88
Perl	Languages	106	2	3.55

Declining

Skill	Category	Mentions (of 2813)	Mentions (of 907)	Δ share (pp)
Spark	Data & Analytics	173	120	-7.08
GCP	Cloud & Infrastructure	199	121	-6.27
Databricks	Data & Analytics	72	78	-6.04
Azure	Cloud & Infrastructure	183	106	-5.18
Scala	Languages	76	66	-4.57

Figure 5: Web dashboard from a full trend view over all 24 companies and categories. This application-level view shows the final output of the scan/extract/normalize/trend pipeline; the controlled systems benchmark in Table 1 uses a fixed 100-posting subset.

the hosted service, a controlled backend would expose queueing, server batching, and accelerator utilization, allowing us to locate the actual saturation point. Prompt caching is also promising because every call shares the same extraction instructions.

Model selection remains an open quality/latency trade-off. A separate exploratory run showed much higher latency under a different backend configuration, but its metadata is insufficient for a clean model comparison. Without a labeled quality set, there is no evidence that a slower model improves the final trend output.

Two application-level optimizations are also promising. First, caching by (`posting_id`, `content_hash`) would avoid re-extracting unchanged descriptions after repeated scans. Second, a cascaded design could use a small model for the first pass and invoke a larger model only when evidence is missing or output confidence is low. Both reduce LLM work without changing the trend interface.

5.3 Limitations

First, only `workers=8` has three repetitions; hosted API measurements should ideally repeat every configuration. Second, the paper evaluates one hosted backend and cannot directly characterize inference-engine utilization. Third, extraction quality lacks precision/recall evaluation on a hand-labeled gold set. Finally, some providers lack reliable `posted_at` values and fall back to `first_seen`, biasing trend windows during the initial collection period.

These limitations define the highest-value next steps: repeat all concurrency settings, compare hosted Gemini with vLLM-on-TPU, and label 30-50 postings for quality evaluation.

The corpus is also not a representative sample of the full labor market. It favors companies with accessible public job endpoints and software/ML-oriented title filters. Company-specific parsers may break when public sites change, and the LinkedIn fallback has rate-limit and

terms-of-service constraints. The reported trends should therefore be interpreted as behavior of the collected corpus, not as universal market statistics.

6. Conclusion

We built an agentic skill-trend pipeline that combines heterogeneous provider tools, concurrent LLM workers, and deterministic validation. On a fixed 100-posting workload, throughput scales by 14.7x at 16 workers while per-call latency remains flat, showing that the hosted backend does not saturate under our tested load. The key application lesson is equally important: an LLM should generate extraction candidates, while deterministic tools enforce output policy and make results reusable. A self-hosted serving comparison and labeled quality benchmark are the remaining steps needed to jointly defend speed and correctness.

Artifact

Code, metrics, and reproduction scripts are in the project repository. Run `bash tools/benchmark_sweep.sh` to reproduce the sweep, `python tools/build_figures.py` to regenerate figures, and `pytest tests/` to run the 101-test suite.

References

1. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023.
2. Google Gemini model documentation.
3. Greenhouse Job Board API.
4. Ashby Job Board API.
5. JobSpy.